```
# DFS using dictionary

graph = {
    '5': ['3', '7'],
    '3': ['2', '4'],
    '7': ['8'],
    '2': [],
    '4': ['8'],
    '8': []
}

visited = set()

def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbour in graph[node]:
            dfs(neighbour)
print(" ---:Task 1 OutPut :---")
print("DFS Traversal:")
dfs('5')


#Task 2







from collections import defaultdict

class Graph:
    def __init__(self):
        self.graph = defaultdict(list)

    def addEdge(self, u, v):
        self.graph[u].append(v)

    def DFSUtil(self, v, visited):
        visited[v] = True
        print(v, end=" ")

        for i in self.graph[v]:
            if not visited[i]:
                self.DFSUtil(i, visited)

    def DFS(self):
        V = len(self.graph)
        visited = [False] * V

        for i in range(V):
            if not visited[i]:
                self.DFSUtil(i, visited)

g = Graph()
g.addEdge(0, 1)
g.addEdge(0, 2)
```

```
g.addEdge(1, 2)
g.addEdge(2, 0)
g.addEdge(2, 3)
g.addEdge(3, 3)

print("\n\n---: Task 2 OutPut :---")

print("DFS Traversal:")
g.DFS()
print("-----\n")



#Lab TAsk 1
# DFS for TSP (Arad to Bucharest)

graph = {
    'Arad': [('Zerind', 75), ('Sibiu', 140), ('Timisoara', 118)],
    'Zerind': [('Oradea', 71)],
    'Oradea': [('Sibiu', 151)],
    'Sibiu': [('Fagaras', 99), ('Rimnicu Vilcea', 80)],
    'Fagaras': [('Bucharest', 211)],
    'Rimnicu Vilcea': [('Pitesti', 97)],
    'Pitesti': [('Bucharest', 101)],
    'Timisoara': [('Lugoj', 111)],
    'Lugoj': [('Mehadia', 70)],
    'Mehadia': [('Drobeta', 75)],
    'Drobeta': [('Craiova', 120)],
    'Craiova': [('Pitesti', 138)],
    'Bucharest': []
}

visited = set()

def dfs_path(node, goal, path, cost):
    path.append(node)

    if node == goal:
        print("Path:", " -> ".join(path))
        print("Total Distance:", cost)
        return True

    visited.add(node)

    for neighbour, distance in graph[node]:
        if neighbour not in visited:
            if dfs_path(neighbour, goal, path, cost + distance):
                return True

    path.pop()
    return False
print("\n ---: Lab Task 1 :---\n")
print("DFS Path from Arad to Bucharest:")
dfs_path('Arad', 'Bucharest', [], 0)
```

```
 ---:Task 1 OutPut :---
DFS Traversal:
5 3 2 4 8 7

---: Task 2 OutPut :---
DFS Traversal:
0 1 2 3 -----
```

```
 ---: Lab Task 1 :---

DFS Path from Arad to Bucharest:
Path: Arad -> Zerind -> Oradea -> Sibiu -> Fagaras -> Bucharest
Total Distance: 607
True
```